

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	_____Dr Sukanya leddala_____
Student Name	_____A M P S SUBRAHMANYA SREEPAD_____
Roll Number	_____2520030152_____
Branch	CSE
Course Outcome	6

Work Done Summary

In this case study, we analyzed a Microservice Call-Graph using Depth First Search (DFS) to identify all downstream services reachable from an API trace.

The work focused on:

1. Representing microservices as graph vertices.
2. Representing service calls as directed edges.
3. Applying DFS traversal on the call graph.
4. Visiting services in alphabetical neighbor order.
5. Identifying all downstream reachable services.
6. Recording DFS traversal order.
7. Computing traversal complexity.
8. Understanding how distributed tracing systems use call-graph traversal for debugging and latency analysis.

Implementation Details

1. Modeled each microservice as a graph node:
 - api
 - auth

- catalog
 - cart
 - inventory
 - ship
 - notif
 - analytics
 - payment
2. Represented service-to-service calls as directed edges.
 3. Built the adjacency list for the 12 directed edges.
 4. Started DFS traversal from:

api

5. Followed alphabetical neighbor order during traversal.
6. Marked services as visited to avoid repeated processing.
7. Recursively explored all reachable downstream services.
8. Generated traversal order based on DFS recursion.
9. Identified all transitively reachable services from the API gateway.
10. Computed DFS complexity:

$O(V+E)O(V+E)O(V+E)$

11. Substituted:

- $V = 9$ services
- $E = 12$ service calls

12. Explained how distributed tracing tools such as Jaeger and Zipkin use DFS-like traversal for dependency analysis and latency debugging.

RESULTS:SCREENSHOTS/Output:

```
Coin Denominations:
[1, 2, 5, 10, 20]

Target Amount:
43

Greedy Selection:
20 + 20 + 2 + 1

Greedy Coin Count:
4

DP Table Computed Successfully.

Minimum Coins Required:
4

Optimal Coin Combination:
[20, 20, 2, 1]

Final Verification:
20 + 20 + 2 + 1 = 43

Coin Change DP Completed Successfully.
```

Conclusion

In this case study, Dynamic Programming was used because it guarantees the optimal minimum number of coins for any denomination system.

The DP approach systematically computed the best solution for every amount from 0 to 43 and reused previously computed subproblems to avoid redundant calculations. A `choice[]` array was also maintained to recover the actual coins used in the optimal solution.

Although the greedy approach worked correctly for the Indian coin system used here, greedy algorithms fail for many non-canonical coin systems. Dynamic Programming

therefore provides a more reliable and general solution for real-world financial and currency optimization problems.

Github Link: <https://github.com/ampssubrahmanyasreepad-svg/DSA-CaseStudies-correct>

Student Signature	Faculty Signature